



---

## Vision & Strategy

*Psilia is the perception and spatial intelligence platform for the edge.*

### The Problem

Building robotics applications and working with perception on edge hardware is harder than it should be. The pain shows up at every stage and each stage has its own friction, its own duct-tape solutions. Nothing covers the full cycle.

1. **Getting started (!!)**. Before you record a single useful frame in the field you've dealt with: OS setup on Jetson, ROS installation, Docker setup, camera drivers, camera calibration, WiFi and SSH configuration, writing launch scripts. Nothing ties it together. Everyone starts from scratch.
2. **Field workflow. (!!)**. In the field you're managing multiple terminals over SSH, manually starting ROS nodes, checking topic rates, starting and stopping recordings. If something breaks you're debugging over SSH with a laptop on your lap. The tooling assumes you're at a desk.
3. **Data sync.** Getting recordings off the device and into a usable location – laptop, NAS, cloud – is another piece of infrastructure every team sets up from scratch. rsync scripts, USB transfers, maybe a cloud upload hook. One more thing you have to wire up and maintain yourself.
4. **Working with data.** Once you have recordings, getting the data into a usable form is annoying. Parsing ROS bags, syncing topics by timestamp, post-processing and slicing recordings for algorithm testing, basic inspection to verify a recording is good. Tools exist but they're fragmented – you still end up writing your own code to bridge the gaps.
5. **Two execution environments.** Embodied AI teams constantly jump between two worlds: the edge runtime (Jetson, ROS, sensors, deployment) and the dev environment (Python, notebooks, cloud, algorithm iteration). The boundary between them creates unnecessary friction. You iterate on an algorithm in a notebook, then have to redeploy to the device to test it in the real world (rewrite for performance, wrap into ROS nodes, rebuild, test on device). That round trip is clumsy. Feedback loops are slow.

6. **Algorithm lock-in (!!)**. Proprietary stacks like the ZED SDK don't give you the hooks to customize or swap algorithms. If you want to change how depth or pose is computed, you're either fighting the SDK or rebuilding from scratch. That's fine until it isn't. Off-the-shelf algorithms work most of the time, however the acceptable failure rate depends entirely on the task. 95% might be fine for a demo, unacceptable for a deployed robot.
  7. **Hardware cost (!!)**. High-quality spatial perception today means expensive proprietary hardware – a ZED Mini starts at \$399 (no global shutter at that price), the ZED X series at \$500+ (with global shutter). That can be a barrier for researchers, developers, and early-stage teams. Off-the-shelf USB stereo cameras like the ELP USB Stereo Camera (~\$100) include a global shutter and are available immediately at a fraction of the cost.
- 

## The Vision

Psilia is built on two convictions:

- **Spatial perception is infrastructure.** Every team building on the edge today owns the entire stack themselves. That's unnecessary. There should be a reliable, hackable foundation teams can build on, so they can focus on what's unique to their mission.
- **Edge-native perception is a distinct category.** Truly autonomous systems need to operate without connectivity, on constrained hardware, in unstructured environments. The dominant trend in AI is *scaling up*: bigger models and more compute. Edge-native perception runs orthogonal to that. It cannot rely on cloud-hosted billion-parameter models. These systems need perception that is fast, lightweight, reliable. This is especially pronounced in fields like agriculture, defense, construction.

We are building the **perception and spatial intelligence platform** for the edge. The layer between raw sensors and the autonomy logic above, as well as the tooling around it. The first conviction informs what we build first. The second informs where it goes – as the platform matures, we replace and strengthen the algorithmic core underneath.

Concretely, that means building toward three things:

1. A robust, reliable spatial perception runtime you can build on.
2. The tooling for fast, frictionless prototyping.
3. The algorithms for high-quality spatial perception at a fraction of the hardware cost.

## Core Bets

- The core architectural bet is a **runtime as a shared abstraction layer**. It connects raw sensors and the autonomy logic above: Plug in a camera, get spatial outputs. Underneath, everything is open and accessible: swap the camera, swap the algorithm, change how things are configured, without fighting the system or rebuilding from scratch. The runtime is hardware-agnostic; the experience stays the same regardless of what's plugged in.
- **Fast iteration as a force multiplier**. Tighter feedback loops produce better outcomes. Teams that can experiment quickly (i.e., record data, iterate on algorithms, redeploy, test) find better solutions faster. Psilia sets up the scaffolding so that cycle can be fast: The runtime handles the device, the tooling handles the data, the dev API bridges back to Python. Less duct-taping, more time on the actual problem.
- **The data flywheel**. Once the runtime is deployed, data accumulates. Owning the sync, storage, and dev tooling around that data ties users deeper into the ecosystem. It also closes the loop between the edge runtime and the dev environment. Data is one of the most valuable assets a robotics team builds. It deserves proper infrastructure, not ad-hoc scripts.
- **Edge-native algorithms are a different discipline**. Data is scarce, compute is limited, the world is ambiguous, and everything runs in real-time. This demands systems that are data-efficient, uncertainty-aware, and compute-adaptive (able to trade accuracy for speed on demand). These constraints run orthogonal to the dominant trend in AI. They demand different algorithms.
- **Algorithms as a retention driver**. As use cases grow, off-the-shelf algorithms hit their limits. Better algorithms keep users on the runtime and decouple performance from hardware cost.

Over time the goal is to be the platform every spatial AI team builds on, regardless of hardware.

## Target, Strategy, and Timing

Large robotics companies and well-funded labs already have internal pipelines for this. They've solved it for themselves. That's not the target.

The target is early-stage startups, small teams, and researchers – even hobbyists – who currently have to own the entire stack themselves. They are feeling the pain today and building the robotics companies of tomorrow. Meeting them early and growing with them is the strategy.

Robotics hardware is getting cheaper. Entry barriers are falling. But cheaper sensors alone don't accelerate the field – the tooling and abstractions around them do.

PyTorch didn't win because it had better algorithms. It won because it made the iteration cycle fast and intuitive. A larger community could experiment, more ideas got tried, and the field moved faster. The community became the moat. That's the model.

Psilia factors out the spatial intelligence layer and makes robust reliable spatial perception the new starting point every team builds from, not the thing they spend months trying to get working reliably.

---

## The First Building Block: Psilia Edge

*Our goal is to be the first thing installed on Jetson. The Docker Desktop for spatial perception.*

*Psilia Edge* is our first product: a **spatial perception runtime** for Jetson, packaged to be easy to install, easy to operate, and easy to build on.

Plug in a stereo camera, run one command, and get reliable depth, pose, and spatial outputs as a stable ROS interface in under 5 minutes. Hardware-agnostic, no ROS wrangling, no bash scripts. Unless you want to. Everything is open and accessible. Swap or extend the underlying algorithms without changing the interface.

A lightweight control UI handles device pairing, network setup, monitoring, recording, and data sync. Recorded data is stored as open MCAP files and accessible via a lightweight Python API. Query topics by time, sync streams, and inspect message statistics in a few lines of code.

### What it does (v0)

Psilia Edge has three parts: the runtime (including core algorithms), a control interface (CLI and web UI), and a minimal dev API.

- Installs a containerized ROS spatial stack on Jetson in minutes
- Runs core spatial algorithms and exposes outputs: `/psilia/depth`, `/psilia/pose`, `/psilia/image`
- Pairs and configures the Jetson from your laptop (SSH, WiFi, hotspot)
- Web-based control UI: control runtime and monitor status, start/stop recording, manage sessions. Works from a phone.
- CLI for pairing, setup, data pull
- Minimal Python API for working with recorded MCAP data offline (topic access, time indexing, basic stats)

### Go-to-market: Camera Bundle

The runtime and core algorithms are free and open source. The v0 go-to-market is a bundled USB stereo camera – cheap off-the-shelf hardware paired with our algorithms and runtime. The goal is not hardware margin. The goal is owning

the first experience end-to-end: plug in the camera, run one command, get spatial outputs. The bundle is distribution and a wedge, not the product. It gets installs, lowers the barrier to a first success, and puts users on the data flywheel from day one.

*Our goal is ZED-quality spatial perception at a fraction of the cost.*

---

## Business Model

The runtime and core algorithms are free and open source. The open runtime gets adoption. Revenue comes from premium services layered on top:

- **Data platform.** Cloud sync, post-processing, inspection, evaluation, annotation. Per-seat pricing (\$200-1,000/month) targeting small teams and early-stage startups. A robotics team already spending \$1,000/month per engineer on cloud compute won't think twice about this.
- **Premium algorithms.** Advanced or domain-specific algorithms beyond the open-source baseline. Open-core model.

Year 1 focus is adoption (200-500 active users) with early data platform revenue. Year 3 target: \$1M+ ARR from 100-200 paying teams.

---

## What Psilia Is Not

Psilia is not a camera product. It overlaps with ZED in what it delivers, i.e. depth, pose, spatial outputs, but the approach is different. ZED's SDK exists to make their hardware useful; it's a means to sell cameras. Psilia's runtime is the product. It's open, hackable, and hardware-agnostic. And it goes further than perception outputs: the goal is to own the full cycle from runtime to data to dev tooling, not just the sensor interface.

Differentiators to ZED/Stereolabs are:

- **Open vs closed** – you can swap algorithms, see what's happening, hack it
- **Hardware-agnostic vs hardware-locked** – Psilia works with any camera.
- **Platform vs product** – Psilia aims to be an ecosystem (runtime + data + dev tooling), ZED is a camera SDK

(TODO: Expand this section. Also address overlap with NVIDIA – Isaac, cuVS-LAM, etc. NVIDIA provides building blocks but not a cohesive runtime; Psilia is the assembled, ready-to-use layer.)

---

## Who It's For

Initially: graduate students and developers, academic research teams, and early-stage startups building embodied AI products. These are the people feeling the pain today and building the products that will need this platform at scale tomorrow.

Graduate students, developers, and academic research teams get adoption – and they build the next wave of startups. Early-stage startups – and the companies they grow into – are the paying customers.

See also: Target, Strategy, and Timing in the Vision section.

## Key Numbers

- 2M+ developers on NVIDIA Jetson
- \$10B+ invested in robotics startups in 2025
- Edge AI market: ~\$25B, growing 22% Compound Annual Growth Rate (CAGR)

References:

- <https://blogs.nvidia.com/blog/2-million-robotics-developers/>
- <https://news.crunchbase.com/venture/aerial-robotics-startup-infravision-seriesb/>
- <https://www.grandviewresearch.com/industry-analysis/edge-ai-market-report>